Simulação de Monte Carlo para Estimativa de π

PERF, Entrega 2 (parcial)

Lucas C. Machado

11 de Novembro de 2026

O que é Monte Carlo

- Família de métodos que estima uma quantidade por amostragem aleatória repetida
- Ideia central: transformar um problema determinístico (ex.: uma área) em uma probabilidade e estimá-la por frequência
- Pela lei dos grandes números, a estimativa converge quando o número de amostras N cresce
- Erro típico proporcional a $N^{-1/2}$: preciso, mas lento de melhorar
- Amostras independentes $\Rightarrow$ método naturalmente paralelizável

Estrutura do projeto

```
monte_carlo/
|-- flake.nix, flake.lock      ambiente reprodutivel (Nix)
|-- Makefile                   orquestra: run, relatorio, test, tgz
|-- src/
|   `-- monte_carlo.cpp        objeto computacional (C++ / OpenMP)
|-- scripts/
|   |-- gera_layout.R          DOE: plano de experimentos (R)
|   |-- executor.sh            executa o plano e consolida o CSV
|   |-- registra_ambiente.sh   captura hardware/software na medicao
|   `-- teste_rapido.sh        mesma semente => mesmo M
|-- dados/
|   |-- input_experimentos.csv plano (ordem, repeticao, threads, N, semente)
|   |-- output_resultados.csv  medicoes brutas (80 linhas)
|   `-- ambiente.txt, lscpu.txt
|-- figuras/                   graficos gerados pela analise
|-- analise.Rmd                analise literal (-> relatorio/analise.pdf)
`-- slides/
    `-- slides.org             esta apresentacao
```

[REPRO] [DSBOX]

Aplicação: estimar π

- Sorteiam-se N pontos uniformes no quadrado $[-1, 1]^2$ (área 4)
- $M =$ pontos com $x^2 + y^2 \leq 1$, dentro do círculo de raio 1 (área π)
- Razão das áreas: $M/N \to \pi/4$, logo $\hat{\pi} = 4M/N$
- Cada casa decimal a mais custa $100\times$ mais pontos
- Pergunta do estudo: como $\hat{\pi}$ (precisão) e o tempo (desempenho) variam com N e com o número de threads?

Implementação e paralelização

- C++ com OpenMP, um binário parametrizado: `./monte_carlo <N> <Threads> <Repeticao> [Semente]`
- Região paralela com `omp for schedule(static)` e `reduction(+:M)`
- Laço *embaraçosamente paralelo*: iterações independentes, sem compartilhar memória
- Um gerador `mt19937` por thread, semeado com `seed_seq` (semente da execução, nº da thread)
- Tempo com `omp_get_wtime()` em volta de toda a região paralela
- O ponto (x, y) não é guardado: sorteado, testado e descartado

DOE: fatorial completo gerado por código

Fator	Níveis
N	10^4, 10^6, 10^8, 10^9
Threads	1, 2, 4, 8
Repetições	5

- $4 \times 4 \times 5 = 80$ execuções
- Plano gerado por `scripts/gera_layout.R` com semente fixa: o *design* também é reprodutível
- Ordem aleatória (coluna `Ordem`) e semente única por execução

[DOE]

Boas práticas de controle experimental

- Aquecimento descartado antes da bateria
- Semente única por execução, registrada no CSV
- Verificação automática dos dados: nº de execuções, $0 \leq M \leq N$, threads reais = pedidas, sementes únicas
- Valor de referência de π conferido entre C++ e R
- Checagem de deriva temporal (tempo $\times$ ordem de execução)
- Carga da máquina registrada no início e no fim

[BOAS]

Ambiente de teste

Item	Valor
CPU	AMD Ryzen 5 5500U (6 núcleos, 12 lógicas)
SO / kernel	Fedora 43 / 7.2.13
Compilador	g++ (GCC) 16.2.0, `-O3 -fopenmp`
Governor	`powersave`
Carga (início / fim)	4,28 / 2,59
RAM	17 GiB

- Registrado automaticamente em `dados/ambiente.txt` *na hora da medição*
- Limitação: máquina não ociosa e frequência variável

[DESC]

Reprodutibilidade: do código ao PDF

- Ambiente fixado com Nix flake (`flake.nix` + `flake.lock`)
- Fluxo em `Makefile`: `make run`, `make relatorio`, `make test`
- Mesmo (N, threads, semente) $\Rightarrow$ mesmo M (testado por `make test`)
- Dados brutos preservados: `make clean` não apaga `dados/`
- Commit do código e hash do `flake.lock` gravados junto com a medição
- Pipeline: `gera_layout.R` $\rightarrow$ `executor.sh` $\rightarrow$ análise $\rightarrow$ PDF

[REPRO]

Ferramentas e programação literal

- Git, Nix, Make: versionamento, ambiente e orquestração
- R (tidyverse, ggplot2): manipulação e gráficos
- R Markdown no relatório atual e Org-mode nestes slides: código, resultado e interpretação no mesmo documento
- Números do texto são gerados dos dados: reexecutar atualiza tudo
- Próximo passo: relatório em Org com o template ACM

[DSBOX] [LITERAL]

O problema mais sério: sementes que se repetiam

- Semente original: `12345 + thread + repeticao`
- Consequência: thread 1 da repetição 1 gerava *exatamente* o mesmo M que a thread 0 da repetição 2
- Repetições não independentes $\Rightarrow$ desvio-padrão subestimado
- Como foi descoberto: reproduzindo a lógica de sementes num teste isolado
- Solução: semente única por execução, definida no plano (R) e repassada ao C++, que deriva um fluxo por thread com `seed_seq`
- Correção sugerida pela IA; a questão das sementes já havia sido levantada na Entrega 1

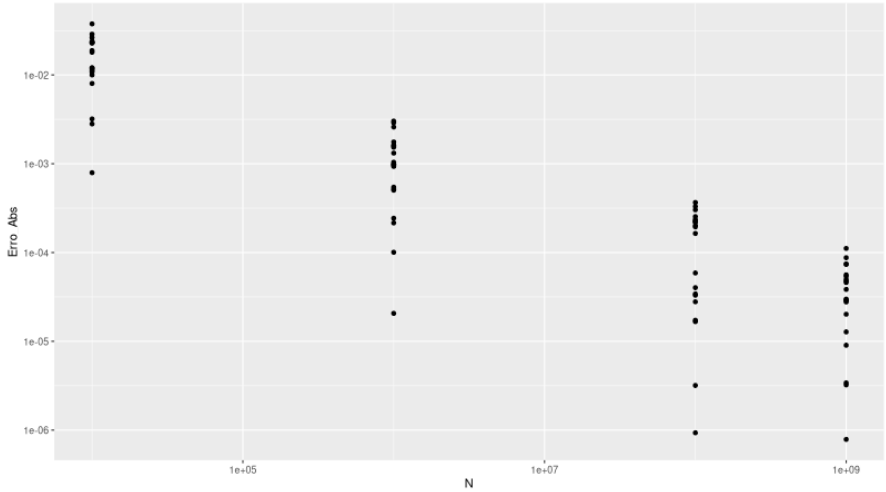
[DIFF]

Outras dificuldades no pipeline

Problema	Solução
Ordem crescente: deriva confundia fatores	Ordem aleatória no plano
`tail + while` com `exit 1` não abortava	Laço sem pipe; saída em `.tmp`
`make clean` apagava os dados medidos	`clean` preserva `dados/`
CSV antigo fazia o `make` pular as medições	Alvo `make rerun`
Locale pt_BR (vírgula) quebrava o `printf`	`LC_ALL=C` no script
Registro do ambiente chamava o `make` em *loop*	Ler flags direto do Makefile

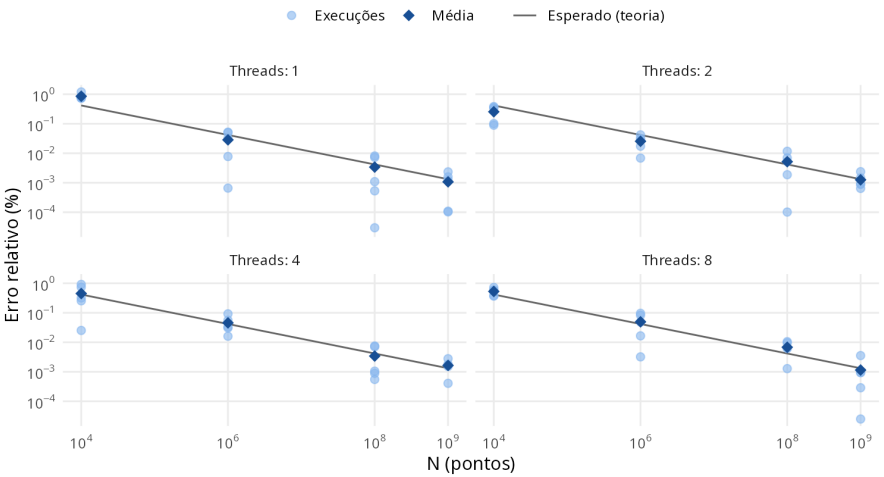
[DIFF]

Precisão de $\hat{\pi}$



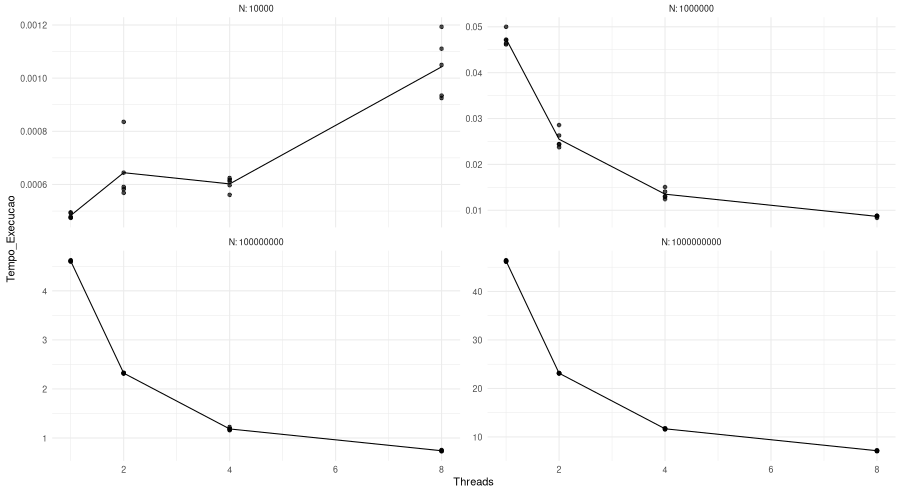
[RESULTS]

Convergência: erro relativo $\times$ N, por número de threads



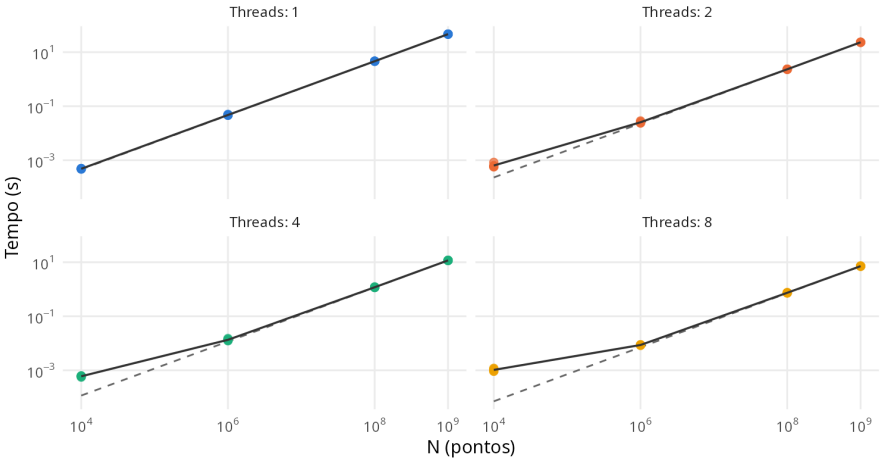
[RESULTS]

Tempo de execução



[RESULTS]

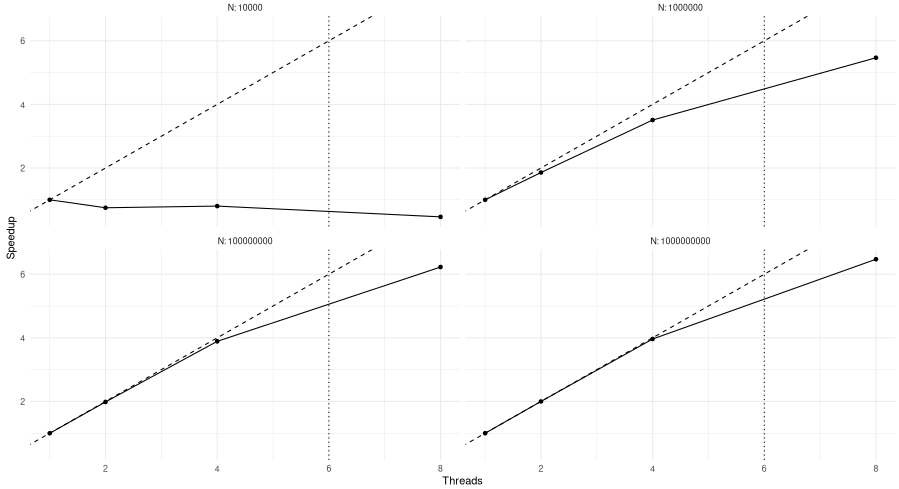
Tempo $\times$ N, por número de threads



Linha sólida: média por N. Tracejado: referência tempo $\propto$ N.

[RESULTS]

Speedup



[RESULTS]

Limitações e próximos passos

- $T(1)$ é o binário OpenMP com 1 thread, não uma versão sequencial pura
- Uma só máquina, com carga não nula; apenas 5 repetições por configuração
- Repetir a medição com a máquina ociosa e tudo commitado
- Registrar as coordenadas (x, y) (com N pequeno) para visualizar o método
- Relatório final em Org com o template ACM

Uso de IA

- Consulta rápida (ex.: regex), síntese de texto (ex.: explicar os critérios em Org) e conversão (ex.: trecho para LaTeX)
- Agente (Claude Code): a partir dos arquivos que já existiam (`monte_carlo.cpp`, `executor.sh`, `gera_layout.R`, `Makefile`, Entrega 1 e enunciado)
  - auditoria do pipeline, correção das sementes, scripts, Makefile e primeira versão da análise
  - ajustes posteriores feitos por mim durante o desenvolvimento
- Rascunho destes slides feito com IA a partir das minhas anotações
- Observação: o modo agente tende a respostas elaboradas demais para consultas simples
- Revisão humana do código e da interpretação: a IA não substituiu o autor mas foi fundamental no processo. Tanto de correção quanto de planejamento e análise.

Progresso e validação

- Validado: estrutura de execução, controle experimental e reprodutibilidade do pipeline
- A validar nos resultados: aderência do erro à curva $N^{-1/2}$
- Em aberto: modelo estatístico completo, baseline sequencial puro, medição em máquina ociosa
- Opinião do autor:
  - A estrutura de execução e análise já sustenta a proposta, com espaço para melhorias
  - O maior aprendizado foi controle experimental, análise de desempenho e reprodutibilidade, mais que o método em si
  - R e Org-mode geraram inércia no início, que diminuiu com o uso. Ainda é preciso melhorar a forma de gerar latex.

[OPIN]